FIAP ON

DATA VIZ & DATA SCIENCE

ANÁLISES DE SÉRIES TEMPORAIS COM PYTHON

14

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Série temporal mal ajustada por uma regressão linear simples 7
Figura 2 – Série temporal ajustada por MA (1).....8
Figura 3 – Exemplo de série não estacionária 9
Figura 4 – Exemplo de série estacionária 10
Figura 5 – ACF sem decaimento.....11
Figura 6 – ACF para uma série estacionária.....11
Figura 7 – Dados ingressos eventos 17
Figura 8 – Tipos das variáveis.....17
Figura 9 – Data ajustada 18
Figura 10 – Dados série temporal inscrições evento.....19
Figura 11 – Gráfico série temporal 20
Figura 12 – Exemplo primeiras diferenças 21
Figura 13 – Gráfico série estacionária.....22
Figura 14 – Gráfico PACF série estudada.....23
Figura 15 – Gráfico ACF série estudada 24
Figura 16 – Resultados ARIMA 26
Figura 17 – Gráfico ACF erros 27
Figura 18 – Gráfico PACF erros 27
Figura 19 – Comparação dados teste e previsões 28

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE QUADROS

Quadro 1 – Ferramentas Python séries temporais. 15

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Importar módulos iniciais e dados.....16
Código-fonte 2 – Dados dos exemplos 16
Código-fonte 3 – Obter os tipos das variáveis.....17
Código-fonte 4 – Ajustar a data.....18
Código-fonte 5 – Separação da data e variável de análise 18
Código-fonte 6 – Obter o gráfico da série temporal.....19
Código-fonte 7 – Execução do teste de Dickey e Fuller.....20
Código-fonte 8 – Aplicação das primeiras diferenças no Python 21
Código-fonte 9 – Análises de estacionariedade 22
Código-fonte 10 – Obtenção do gráfico PACF 23
Código-fonte 11 – Obtenção do gráfico ACF.....24
Código-fonte 12 – Divisão treino e teste 25
Código-fonte 13 – Modelo ARIMA treinado.....25
Código-fonte 14 – Obtenção dos gráficos ACF e PACF dos erros.....26
Código-fonte 15 – Obtenção das previsões do modelo.....28
Código-fonte 16 – Obtenção das medidas de avaliação amostra teste 29

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

1 ANÁLISES DE SÉRIES TEMPORAIS COM PYTHON 6
1.1 Introdução 6
1.2 Principais conceitos sobre análise de séries temporais 7
1.3 Conceito de estacionariedade 9
1.4 Procedimentos para análise de séries temporais: modelo ARIMA 12
1.4.1 Integração 12
1.4.2 Modelo AR.....12
1.4.3 Modelo MA 13
1.4.4 Modelo ARIMA 13
1.4.5 Avaliação dos modelos.....14
2 APLICAÇÕES PRÁTICAS EM SÉRIES TEMPORAIS 15
2.1 Principais funções no Python 15
2.2 Dados e exemplos.....16
2.3 Preparação dos dados 17
2.4 Aplicação do modelo ARIMA.....19
2.4.1 Identificar a ordem de integração I(d).....19
2.4.2 Identificar a ordem AR(p) 23
2.4.3 Identificar a ordem MA(q) 24
2.5 Treinamento do modelo ARIMA 25
2.6 Avaliação do modelo 28
ATIVIDADE TREINO – ANÁLISE DE SÉRIES TEMPORAIS.....30
REFERÊNCIAS.....31

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1 ANÁLISES DE SÉRIES TEMPORAIS COM PYTHON

1.1 Introdução

A expressão “esqueça o passado e viva o presente” não está adequada ao tópico de análise de séries temporais. Nessa área de estudo, a observação do passado é fundamental para verificação de tendências e projeções de metas. Este tópico não é novo, porém tem completa aderência aos objetivos relacionados ao *Data Analytics*.

O estudo de séries temporais possui importância em diversas áreas, como: economia, negócios, engenharia, biologia, meteorologia e ciências sociais, uma vez que os modelos obtidos podem descrever o mecanismo gerador dos dados do evento estudado.

Deste modo, o mecanismo gerador é analisado estatisticamente e representado matematicamente por equações que descrevem o quanto o passado de um evento pode influenciar nas realizações futuras.

Box, Jenkins e Reinsel (2008) apresentam cinco possíveis aplicações para séries temporais:

1. Projeção de valores futuros de uma série.
2. Definição da sequência de entradas e saídas de sistemas dinâmicos.
3. Realizar análises de intervenções, como políticas públicas ou políticas de preços.
4. Verificar a correlação dinâmica entre diversas variáveis no tempo.
5. Analisar controle de sistemas e potencial de desvios.

O objetivo deste texto é dar foco na projeção de valores futuros, uma vez que possui aplicação comum nas tarefas diárias de um analista de dados. Nos próximos tópicos, apresentaremos a metodologia clássica proposta por Box e Jenkins para realizar previsões de séries temporais. Conceitos como estacionariedade e integração serão discutidos e o bem conhecido modelo ARIMA *(Autoregressive Integrated Moving Average)* será descrito.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.2 Principais conceitos sobre análise de séries temporais

Conforme Box, Jenkins e Reinsel (2008), uma série temporal é uma sequência de observações ordenadas no tempo. Podemos verificar uma gama de aplicações em dados nesse formato, como: vendas mensais/diárias de um produto, número de acidentes semanais em uma fábrica ou trânsito, número de acessos em um site, quantidade de buscas semanais de um determinado tópico em um sistema de buscas.

Graficamente, uma série temporal é representada com o eixo x, o tempo e o eixo y, a medida observada. Considerando o contexto do modelo de Regressão Linear, bastaríamos ajustar uma reta em que a variável dependente seria a medida observada e a variável independente, o tempo. Porém, o resultado do modelo seria como a Figura “Série temporal mal ajustada por uma regressão linear simples”.

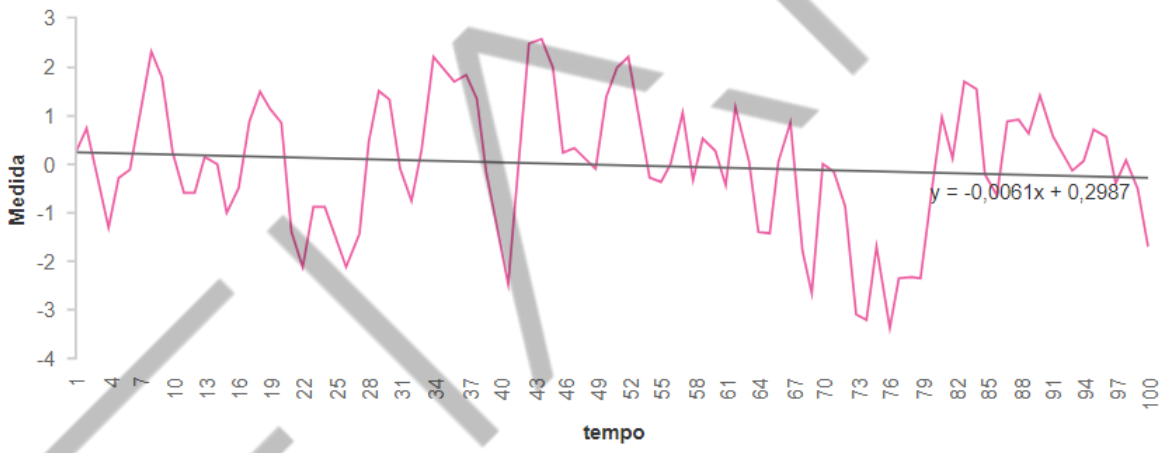

Figura 1 – Série temporal mal ajustada por uma regressão linear simples
Fonte: Elaborado pelo autor (2021)

Conforme Shumway e Stoffer (2017), o modelo clássico de regressão é insuficiente para explicar toda a dinâmica existente em uma série temporal, uma vez que não captura as estruturas adicionais relacionadas ao passado da série. Uma dessas dinâmicas é o fenômeno da possível correlação existente entre os dias passados, possibilitando estimar um dos modelos mais conhecidos, o ARIMA.

O modelo ARIMA se tornou bastante utilizado devido à sua simplicidade de estimação. O método é formado por três partes: AR *(Autoregressive),* I *(Integrated)* e MA *(Moving Average).* A captura dessas três estruturas possibilita um melhor ajuste da série e, consequentemente, a capacidade de previsão.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Conforme é possível observar na Figura “Série temporal ajustada por modelo MA(1)”, a mesma série da Figura “Série temporal mal ajustada por uma regressão linear simples” foi ajustada por um modelo denominado MA(1). Dessa forma, obtendo um melhor ajuste do modelo da série estudada.

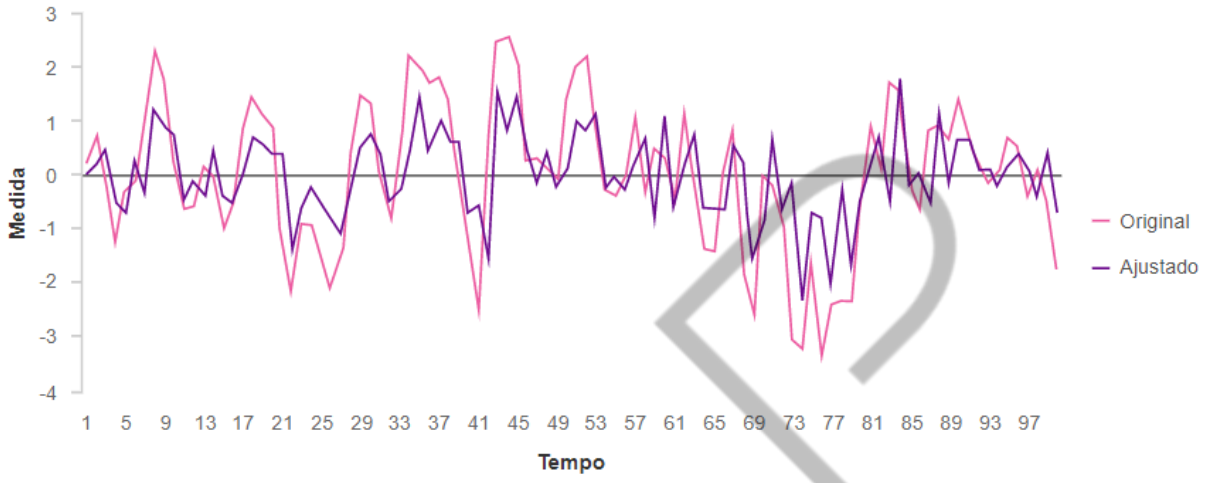

Figura 2 – Série temporal ajustada por MA(1)
Fonte: Elaborado pelo autor (2021)

De acordo com Shumway e Stoffer (2017), para ajustar um modelo ARIMA são necessários seis passos:

1. Plotar os dados.
2. Verificar a necessidade de transformar os dados (verificar estacionariedade).
3. Identificar as ordens de dependência.
4. Estimar os parâmetros.
5. Diagnosticar a qualidade do modelo.
6. Escolher o modelo.

Antes de entrar em detalhes sobre o modelo ARIMA, precisamos detalhar o conceito de estacionariedade.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.3 Conceito de estacionariedade

Formalmente, uma série temporal é considerada como um processo estocástico, uma vez que esse processo é uma sequência ordenada de $x(.,t)$, sendo $x(.,.)$ uma função de filtro e t um conjunto de índices pertencentes a $\mathbb{R}$, que também pode ser representado pelo tempo. Para que esse processo seja estável e, portanto, possibilitar que seja estudado, algumas restrições precisam ser realizadas: homogeneidade temporal e memória do processo.

A primeira restrição remete ao termo “estacionariedade”, que, de acordo com Box, Jenkins e Reinsel (2008), se refere ao equilíbrio estatístico, isto é, apesar de os valores da série variarem no tempo, as propriedades probabilísticas não mudam no tempo.

Deste modo, séries estacionárias possuem média e variância constantes. Nas Figuras “Exemplo de série não estacionária” e “Exemplo de série estacionária”, são apresentados exemplos de uma série não estacionária e estacionária.

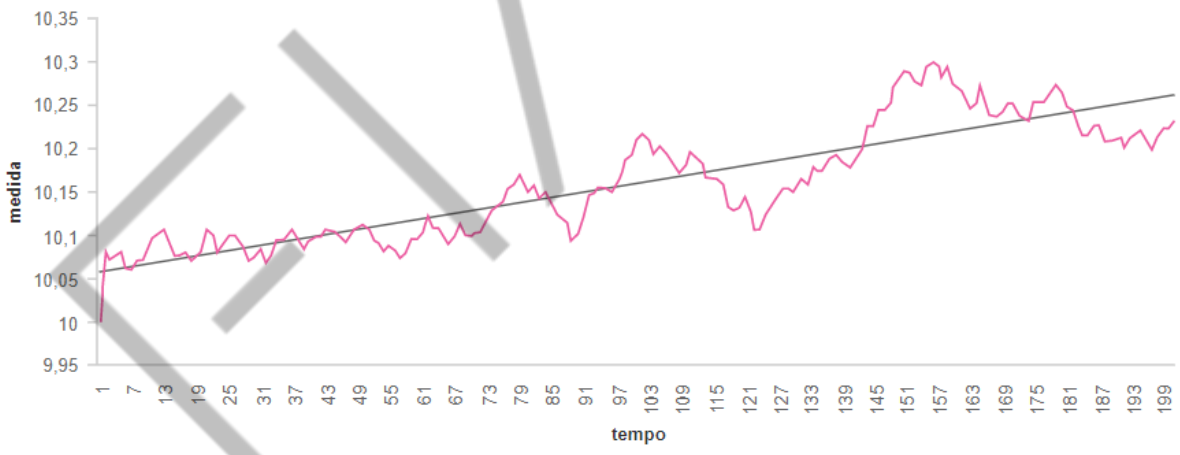

Figura 3 – Exemplo de série não estacionária
Fonte: Elaborado pelo autor (2021)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

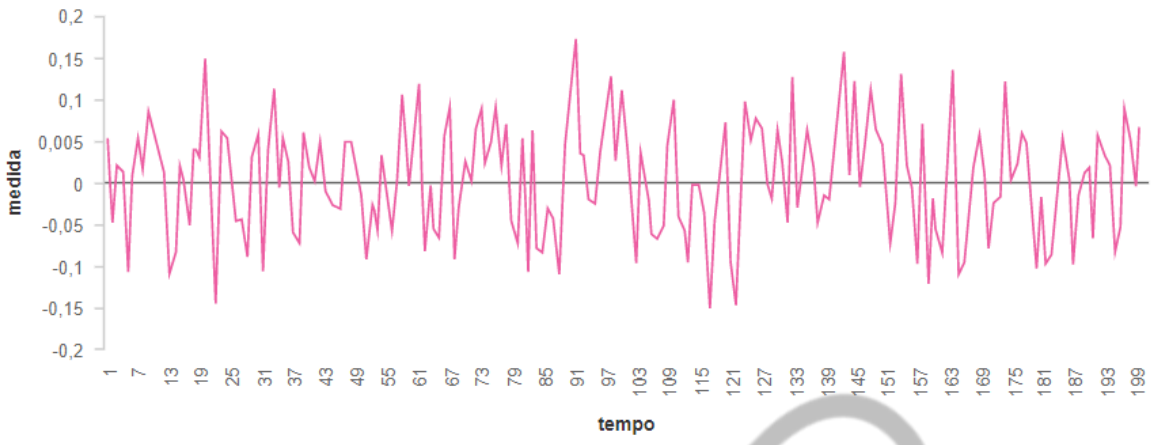

Figura 4 – Exemplo de série estacionária
Fonte: Elaborado pelo autor (2021)

Conforme podemos observar, na primeira figura temos tanto média quanto variância crescentes no tempo, enquanto na segunda observamos tanto média quanto variância constantes no tempo.

A segunda restrição se refere à memória do processo estocástico. Nesse caso, a correlação entre as defasagens da medida no tempo precisa ser finita com o passar do tempo. Esse tipo de restrição pode ser analisado por meio do gráfico de Autocorrelação (ACF – *Autocorrelation Function)* e Autocorrelação Parcial (PACF – *Partial Autocorrelation Function).*

Esses gráficos apresentam as correlações entre os períodos x_t e x_{t-1}. Para que o processo gerador da série temporal possa ser estimado, as correlações entre esses períodos precisam decair com o tempo. Caso isso não ocorra, temos uma série não estacionária.

Nas Figuras “ACF sem decaimento” e “ACF para uma série estacionária”, são apresentados um gráfico ACF que apresenta o comportamento de uma série não estacionária e outro que demonstra claramente o decaimento exponencial da correlação temporal.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

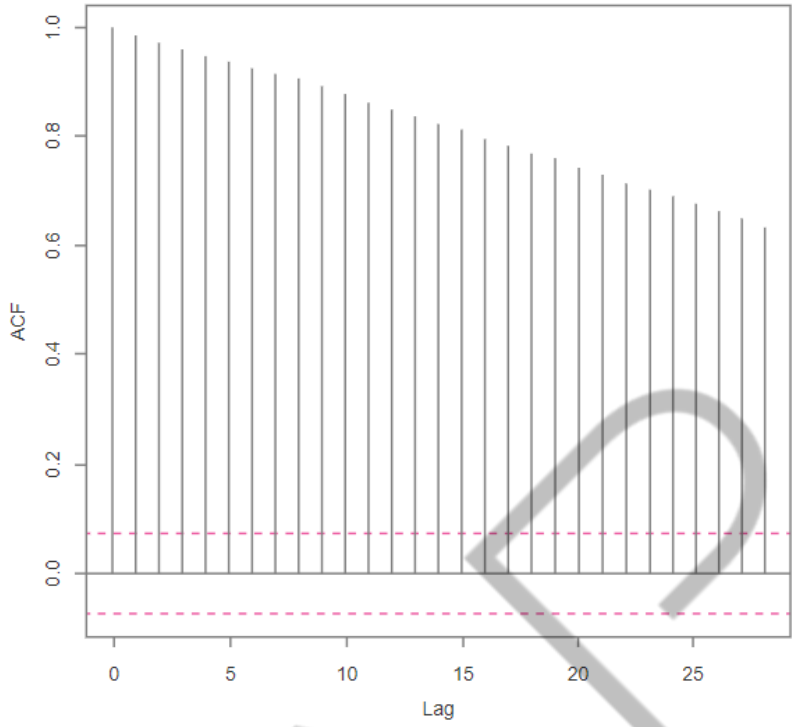

Figura 5 – ACF sem decaimento
Fonte: Elaborado pelo autor (2021)

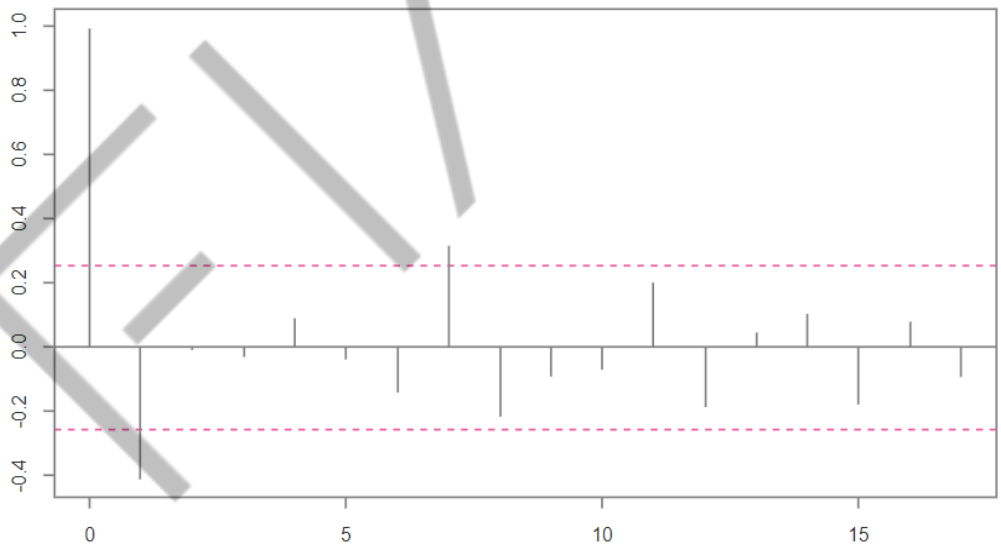

Figura 6 – ACF para uma série estacionária
Fonte: Elaborado pelo autor (2021)

Conforme pode ser observado na Figura “ACF sem decaimento”, as correlações entre as defasagens não decaíram mesmo após 25 períodos. No caso da Figura “ACF para uma série estacionária”, observam-se baixas correlações entre as defasagens, comportamento típico de uma série estacionária.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Caso uma série temporal não seja estacionária, podemos realizar transformações para obter a estacionariedade e, assim, possibilitar a estimação dos parâmetros e previsão das observações futuras. Nos próximos tópicos, o processo ARIMA será detalhado, demonstrando claramente como obter a estacionariedade e estimar os parâmetros.

1.4 Procedimentos para análise de séries temporais: modelo ARIMA

Conforme foi mencionado anteriormente, um processo ARIMA é composto por três partes: AR *(Autoregressive),* I *(Integrated)* e MA *(Moving Average).* O componente I (integração) está relacionado ao processo de manter a estacionariedade da série, enquanto os componentes AR e MA estão relacionados aos parâmetros de ajuste dos modelos.

1.4.1 Integração

A integração da série é importante para conseguir a estacionariedade da série. Por exemplo, caso o processo y_t seja não estacionário, podemos realizar a transformação $\Delta y_t = y_t - y_{t-1}$, denominada de diferenciação. Caso esse novo processo seja estacionário, podemos afirmar que y_t é uma série integrada de primeira ordem, ou seja, $y_t \sim I(1)$.

Caso mais diferenciações sejam necessárias, podemos definir maiores ordens de integração, nesse caso, teremos $\Delta^d y_t \sim I(d)$. Geralmente em séries econômicas, como preços de ações e vendas de produtos, dificilmente serão observadas ordens de integração maiores que 2.

1.4.2 Modelo AR

Em um modelo Autorregressivo, a variável dependente é explicada exclusivamente por seus valores passados. A ordem de um modelo AR é definida pela quantidade de parâmetros utilizados na estimação. Nas expressões 1 e 2, são apresentados modelos AR(1) e AR(p), respectivamente.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

$$y_t = \phi_0 + \phi_1 y_{t-1} + \varepsilon_t. \tag{1}$$

$$y_t = \phi_0 + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t. \tag{2}$$

Em que, ϕ_i são os parâmetros e ε_t é o resíduo do modelo.

1.4.3 Modelo MA

Um modelo de Médias Móveis busca explicar a variável dependente utilizando seus resíduos passados. Assim, como no modelo AR, a ordem de um modelo MA é definida pela quantidade de parâmetros utilizados na estimação. Nas expressões 3 e 4, são apresentados modelos MA(1) e MA(q), respectivamente.

$$y_t = \theta_0 + \theta_1 \varepsilon_{t-1} + \varepsilon_t. \tag{3}$$

$$y_t = \theta_0 + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_p \varepsilon_{t-p} + \varepsilon_t. \tag{4}$$

Em que, θ_i são os parâmetros e ε_t é o resíduo do modelo.

1.4.4 Modelo ARIMA

Por meio da combinação dos três modelos apresentados anteriormente, podemos definir o modelo Autorregressivo Integrado de Média Móvel. Nesse método, a série é ajustada para manter a estacionariedade, conforme a ordem de integração $I(d)$, e os parâmetros são ajustados de acordo com as ordens autorregressivas $AR(p)$ e as ordens de médias móveis $MA(q)$. Nas expressões 5 e 6, são apresentados os modelos $ARIMA(1,1,1)$ e $ARIMA(p,d,q)$, respectivamente.

$$\Delta y_t = \mu + \phi_1 y_{t-1} + \theta_1 \varepsilon_{t-1} + \varepsilon_t. \tag{5}$$

$$\Delta^d y_t = \mu + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \theta_1 \varepsilon_{t-1} + \cdots + \theta_p \varepsilon_{t-p} + \varepsilon_t. \tag{6}$$

Em que, ϕ_i e θ_i são os parâmetros de ajuste, μ é o drift e ε_t é o resíduo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.4.5 Avaliação dos modelos

Assim como foi apresentado nos casos da análise de regressão, para avaliar modelos de séries temporais, também podemos dividir a amostra de análise para treinamento e teste. A diferença para os casos anteriores é que a divisão é mediada por uma data de referência.

Por exemplo, caso tenhamos uma série de vendas no período entre 01/01/2000 e 01/01/2010, podemos definir uma data de referência como 01/01/2009. Dessa forma, a amostra de treinamento poderá compreender o período entre 01/01/2000 e 31/12/2008, e a amostra de teste, o período entre 01/01/2009 e 01/01/2010.

As métricas para avaliação dos modelos de previsão de séries temporais são essenciais para verificar a qualidade dos parâmetros estimados. No caso de séries temporais, as métricas de avaliação na amostra de teste são MAE *(Mean Absolute Error),* MAPE *(Mean Absolute Percentage Error)* e RMSE *(Root Mean Squared Error).*

Para o caso das séries temporais, o RMSE para h períodos à frente é demonstrado na expressão 7:

$$RMSE = \sqrt{\frac{\sum_{t=1}^{T+h}(\hat{y}_{t+h} - y_{t+h})^2}{T+h}}. \tag{7}$$

A métrica MAE possui o mesmo princípio da medida RMSE, porém apresenta o erro em valores absolutos, conforme é apresentado na expressão 8:

$$MAE = \frac{\sum_{t=1}^{T+h}|\hat{y}_{t+h} - y_{t+h}|}{T+h}. \tag{8}$$

Em alguns casos, pode ser interessante demonstrar os resultados em formato de percentual do erro. Nessas situações, a métrica MAPE, expressão 9, pode ser uma boa escolha:

$$MAPE = \left(\frac{\sum_{t=1}^{T+h}\frac{|\hat{y}_{t+h} - y_{t+h}|}{y_{t+h}}}{T+h}\right) \cdot 100. \tag{9}$$

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 APLICAÇÕES PRÁTICAS EM SÉRIES TEMPORAIS

Nos próximos tópicos, serão trabalhados em mais detalhes os conceitos citados anteriormente. Além disso, aplicaremos algumas das ferramentas como exemplo prático no Python.

Desse modo, vamos conhecer as principais funções, os dados e aplicar os modelos de séries temporais.

2.1 Principais funções no Python

Como ferramentas necessárias, os módulos Numpy e Pandas serão as principais bibliotecas do Python novamente. As bibliotecas *matplotlib* e *seaborn* serão novamente utilizadas para análise gráfica.

No caso da análise de séries temporais, trabalharemos com os módulos *scikit-learn* e *statsmodels*. O módulo *statsmodels* facilita a execução e leitura dos resultados dos algoritmos; o *scikit-learn* é uma das ferramentas mais utilizadas, será usada para obter outras medidas de avaliação dos resultados.

No Quadro “Ferramentas Python séries temporais”, apresentamos um resumo das principais funções que serão utilizadas nos exemplos.

Módulos	Funções	Aplicações
Matplotlib	Várias funções serão utilizadas	Análise gráfica
Seaborn	Várias funções serão utilizadas	Análise gráfica
Numpy	Várias funções serão utilizadas	Análise numérica
	sqrt	Função para obter a raiz quadrada
Pandas	Várias funções serão utilizadas	Leitura e tratamento de dados
Scikit-Learn	mean_absolute_error	Dividir os dados amostra treino e teste
	mean_absolute_percentage_error	Aplicar modelo de regressão
	mean_squared_error	Obter o erro médio quadrático
Statsmodels	adfuller	Realizar teste de Dickey e Fuller – verificar estacionariedade
	plot_acf	Obter gráfico de autocorrelação
	plot_pacf	Obter gráfico de autocorrelação parcial
	ARIMA	Estimação do modelo ARIMA
	summary	Sumário dos resultados dos modelos ARIMA

Quadro 1 – Ferramentas Python séries temporais
Fonte: Elaborado pelo autor (2021)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.2 Dados e exemplos

Novamente utilizaremos o *Colab* digitando no navegador https://colab.research.google.com/, criaremos um novo notebook e iniciaremos as análises. Nos exemplos a seguir, utilizaremos os dados “dados_vendas_evento.csv”. O objetivo é estudar e prever o comportamento futuro de compras de ingressos de um determinado evento.

Os códigos para importar os módulos e carregar os dados podem ser vistos no Código-fonte “Importar módulos iniciais e dados”. Juntamente com os códigos apresentados neste texto, também é possível analisar os códigos de forma mais detalhada no arquivo “cap8_exemplos.ipynb” disponibilizado.

```
# gerais
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# series temporais
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_percenta
ge_error, mean_absolute_error

# carregar dados
dados_eventos = pd.read_csv('dados_vendas_evento.csv')
```

Código-fonte 1 – Importar módulos iniciais e dados
Fonte: Elaborado pelo autor (2023)

No Código-fonte “Dados dos exemplos”, podemos observar como obter as cinco primeiras linhas do *dataset* que será utilizado.

```
# dados gastos
dados_eventos.head()
```

Código-fonte 2 – Dados dos exemplos
Fonte: Elaborado pelo autor (2021)

Na Figura “Dados ingressos evento”, podemos observar as cinco primeiras linhas do *dataset*. Temos quatro variáveis, porém utilizaremos as colunas “data_venda” e “qtd_inscricoes”. No caso, o passado da variável “qtd_inscricoes” será utilizado para projetar as vendas futuras.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

data_venda	data_evento	qtd_inscricoes	ticket
2017-02-10	2017-09-13	7	1960.357143
2017-03-23	2017-09-13	4	1621.750000
2017-03-27	2017-09-13	11	1533.583333
2017-04-10	2017-09-13	6	2079.166667
2017-04-11	2017-09-13	4	1871.250000

Figura 7 – Dados ingressos eventos
Fonte: Elaborado pelo autor (2021)

2.3 Preparação dos dados

Antes de realizar o treinamento do modelo, precisamos preparar os dados. O primeiro ponto a ser observado é o tipo das variáveis; no Código-fonte “Obter os tipos das variáveis” mostramos como obter os tipos de cada coluna. Os resultados são vistos na Figura “Tipos das variáveis”.

```
# tipos
dados_eventos.dtypes
```

Código-fonte 3 – Obter os tipos das variáveis
Fonte: Elaborado pelo autor (2021)

```
data_venda          object
data_evento         object
qtd_inscricoes       int64
ticket             float64
dtype: object
```

Figura 8 – Tipos das variáveis
Fonte: Elaborado pelo autor (2021)

Podemos observar que as colunas “data_venda” e “data_evento” são do tipo *object* (texto), “qtd_inscricoes” do tipo inteiro *(int64)* e tíquete do tipo razão (float64). O principal problema, neste caso, está nas colunas de datas; dessa forma, torna-se importante realizar os ajustes para deixar no formato necessário.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

No Código-fonte “Ajustar a data”, demonstramos a forma como ajustamos a data utilizando o comando do Pandas “pd.to_datetime”, após rodar a função “.dtypes”, teremos a coluna ajustada conforme Figura “Data ajustada”.

```
# ajustar data
dados_eventos['data_venda'] = pd.to_datetime(dados_eventos['data_venda
'])

# tipos
dados_eventos.dtypes
```

Código-fonte 4 – Ajustar a data
Fonte: Elaborado pelo autor (2021)

Coluna ajustada

data_venda	datetime64[ns]
data_evento	object
qtd_inscricoes	int64
ticket	float64
dtype: object	

Figura 9 – Data ajustada
Fonte: Elaborado pelo autor (2021)

Outro ajuste importante no Python é separar as variáveis de data e a variável que utilizaremos para previsão. A variável “data_venda” será usada como índice do Pandas e a coluna “qtd_inscricoes” será o objetivo de análise. No Código-fonte “Separação da data e variável de análise”, apresentamos o processo de seleção das colunas e ajuste da coluna data para índice. Na Figura “Dados série temporal inscrições evento”, temos os resultados.

```
# seleção das colunas
dados_analises = dados_eventos[['data_venda', 'qtd_inscricoes']]

# data para indice
dados_analises.set_index('data_venda',inplace=True)

# data para indice
dados_analises.head()
```

Código-fonte 5 – Separação da data e variável de análise
Fonte: Elaborado pelo autor (2021)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	qtd_inscricoes
data_venda	
2017-02-10	7
2017-03-23	4
2017-03-27	11
2017-04-10	6
2017-04-11	4

Figura 10 – Dados série temporal inscrições evento
Fonte: Elaborado pelo autor (2021)

2.4 Aplicação do modelo ARIMA

Nos próximos tópicos, treinaremos o modelo ARIMA, seguiremos as etapas básicas para treinamento do algoritmo: identificar a ordem de integração, identificar ordem AR(p), identificar ordem MA(q), estimar os parâmetros e avaliar o modelo.

2.4.1 Identificar a ordem de integração I(d)

O primeiro passo para identificar a ordem de integração é realizar uma análise gráfica. No Código-fonte “Obter o gráfico da série temporal”, executamos o processo no Python para obter o gráfico. Na Figura “Gráfico série temporal”, temos os resultados.

```
# seleção das colunas
dados_analises.plot(legend = None)
```

Código-fonte 6 – Obter o gráfico da série temporal
Fonte: Elaborado pelo autor (2021)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

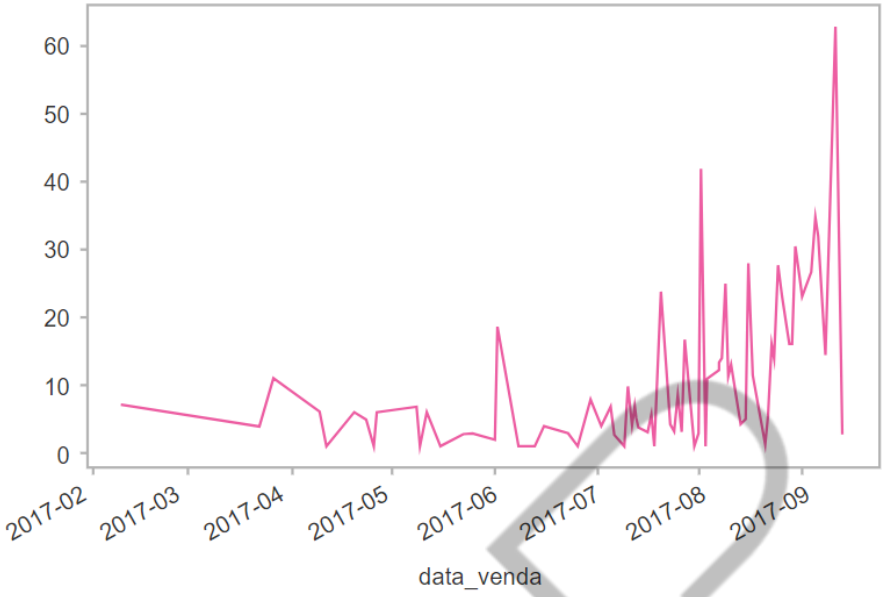

Figura 11 – Gráfico série temporal
Fonte: Elaborado pelo autor (2021)

Observamos que a série está com tendência de alta, propriedades relativas a uma série não estacionária. Para termos certeza da não estacionariedade, precisamos aplicar um teste de hipótese; no caso, a hipótese nula e alternativa serão:

$$H_0\text{: A série não é estacionária}$$

$$H_a\text{: A série é estacionária}$$

Essas hipóteses são avaliadas pelo teste de Dickey e Fuller. No Python, a realização do teste é bastante simples; no Código-fonte "Execução do teste de Dickey e Fuller", apresentamos como realizar o teste.

```
# teste
teste_adf = adfuller(dados_analises['qtd_inscricoes'])
# valor da estatistica
print('Estatística ADF: {:.4}'.format(teste_adf[0]))
# p-value
print('p-value: {:.4}'.format(teste_adf[1]))
```

Código-fonte 7 – Execução do teste de Dickey e Fuller
Fonte: Elaborado pelo autor (2021)

Como resultados, observamos os valores de 0,9259 para a estatística ADF e 0,9934 para o *p-value*. Como o *p-value* foi maior que 0,05, não podemos rejeitar a hipótese nula; desta forma, a série analisada não é estacionária. Diante disso, precisaremos realizar algumas transformações para obter a estacionariedade.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A técnica mais utilizada para ajustar a não estacionariedade é denominada “primeiras diferenças”. Basicamente, pode-se subtrair o valor atual pelo valor do dia anterior, torna-se muito parecido com tentar obter a taxa de variação diária. Realizar esse processo no Python é bastante simples, no Código-fonte “Aplicação das primeiras diferenças no Python”, apresentamos a forma de obtenção, e na Figura “Exemplo primeiras diferenças”, temos o resultado.

```
# primeiras diferencas
dados_analises.diff().head()
```

Código-fonte 8 – Aplicação das primeiras diferenças no Python
Fonte: Elaborado pelo autor (2021)

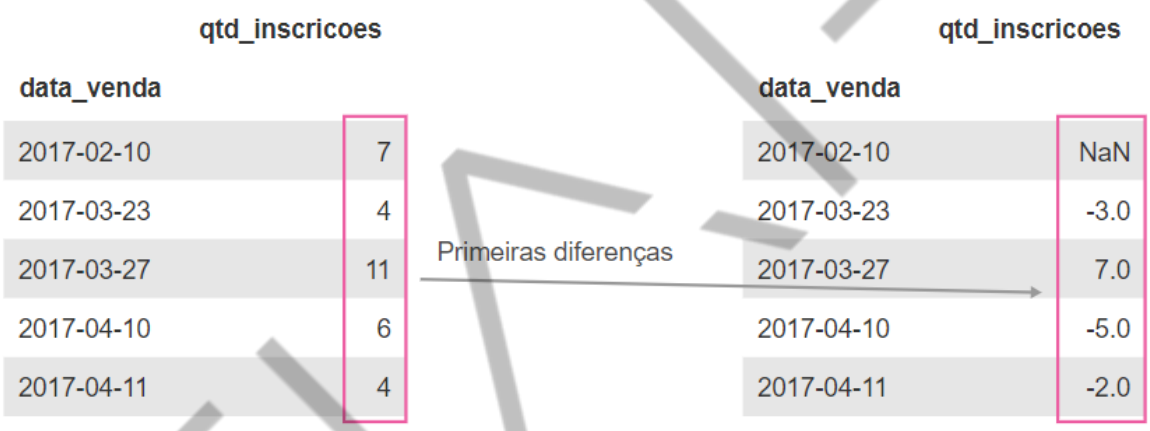

Figura 12 – Exemplo primeiras diferenças
Fonte: Elaborado pelo autor (2021)

Conforme pode ser observado, a técnica é bastante simples. Basicamente, em vez de analisar a série pura, podemos utilizar a série transformada. Para termos certeza de que a série está estacionária, podemos novamente realizar o teste de Dickey-Fuller e observar o novo gráfico.

No Código-fonte “Análises de estacionariedade”, realizamos o teste “adfuller” e geramos o novo gráfico. Na Figura “Gráfico série estacionária”, temos o gráfico da série ajustada.

```
# utilizamos a tecnica das primeiras diferencas
teste_adf = adfuller(dados_analises['qtd_inscricoes'].diff().dropna())

# valor da estatistica
print('Estatística ADF: {:.4}'.format(teste_adf[0]))

# p value
print('p-value: {:.4}'.format(teste_adf[1]))
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# grafico
dados_analises.diff().plot(legend = None)
```

Código-fonte 9 – Análises de estacionariedade
Fonte: Elaborado pelo autor (2021)

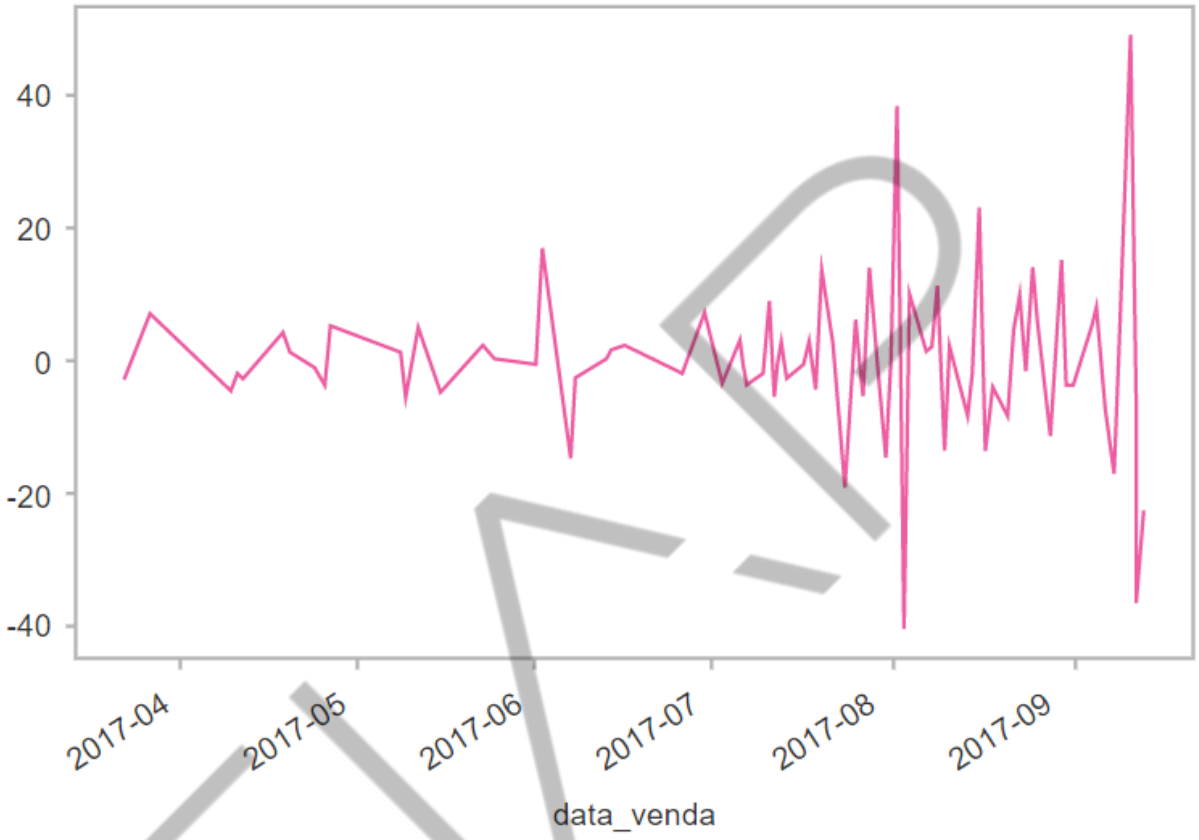

Figura 13 – Gráfico série estacionária
Fonte: Elaborado pelo autor (2021)

Podemos observar no gráfico que a série se manteve constante na média, com alguns topos e fundos. Os resultados do teste de estacionariedade foram ADF = -6,216 e valor p = 5,377e-08; esses valores indicam que podemos rejeitar a hipótese nula; ou seja, a série se tornou estacionária após a transformação.

Um ponto interessante é a possibilidade de realizar esse processo várias vezes até obter a estacionariedade. Em nosso caso, foi necessária somente uma única vez, portanto, a ordem de integração será 1, isto é, I(1). Caso precisássemos realizar novamente, a ordem seria I(2).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.4.2 Identificar a ordem AR(p)

A ordem de integração foi identificada – no caso, I(1) –, agora o objetivo é identificar as ordens AR. Essa avaliação ocorre de forma visual pela análise do gráfico de autocorrelação parcial (PACF). Quando se observam valores altos de correlação parcial, existe o indicativo daquele período explicar os valores futuros de uma variável.

No Código-fonte “Obtenção do gráfico PACF”, podemos executar a análise no Python. Na Figura “Gráfico PACF série estudada”, analisamos o gráfico de autocorrelação parcial da série de tempo.

```
# gráfico pacf
fig = plt.figure(figsize=(12,8))
ax1 = fig.add_subplot(211)
fig = plot_pacf(dados_analises['qtd_inscricoes'].diff().dropna(), lags
=15, ax=ax1)
```

Código-fonte 10 – Obtenção do gráfico PACF
Elaborado pelo autor (2021)

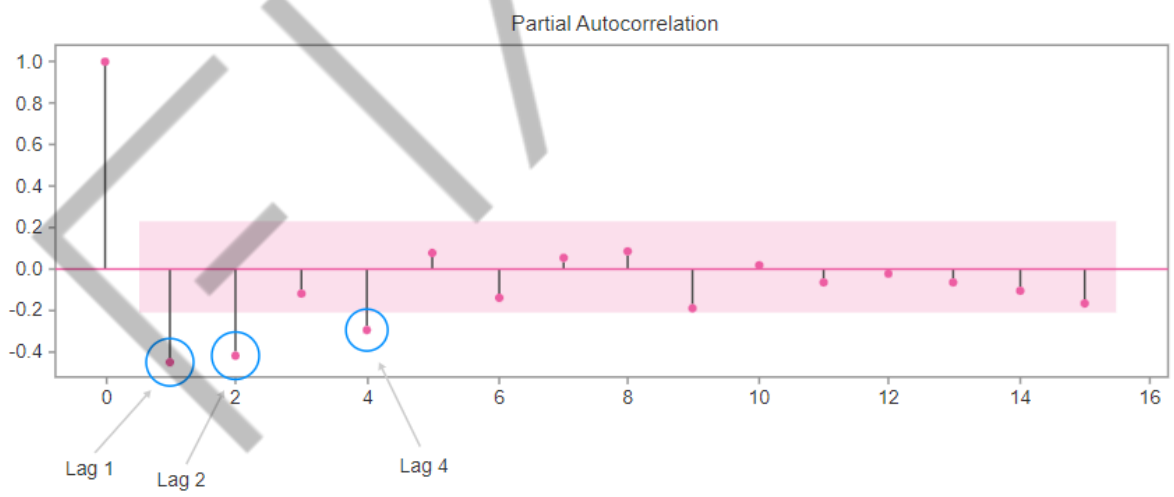

Figura 14 – Gráfico PACF série estudada
Fonte: Elaborado pelo autor (2021)

A análise do gráfico é bastante simples, caso algum ponto esteja fora da região marcada, será uma ordem para o AR. No caso, observamos três defasagens (defasagens 1, 2 e 4) fora da linha tracejada. Portanto, temos uma ideia de que a parte AR terá ordem de até 4 (maior *lag* foi 4).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.4.3 Identificar a ordem MA(q)

O quebra cabeça está começando a ser montado. Como ordem de integração, temos I(1) e ordem de autorregressão AR(4), falta definir a ordem de médias móveis. O processo é bastante similar ao AR, a diferença ocorre porque, em vez de avaliar o gráfico PACF, analisaremos o gráfico de autocorrelação (ACF).

No Código-fonte “Obtenção do gráfico ACF”, podemos executar a análise no Python. Na Figura “Gráfico ACF série estudada”, analisamos o gráfico de autocorrelação da série de tempo.

```
# gráfico acf
fig = plt.figure(figsize=(12,8))
ax1 = fig.add_subplot(211)
fig = plot_acf(dados_analises['qtd_inscricoes'].diff().dropna(), lags=
15, ax=ax1)
```

Código-fonte 11 – Obtenção do gráfico ACF
Fonte: Elaborado pelo autor (2021)

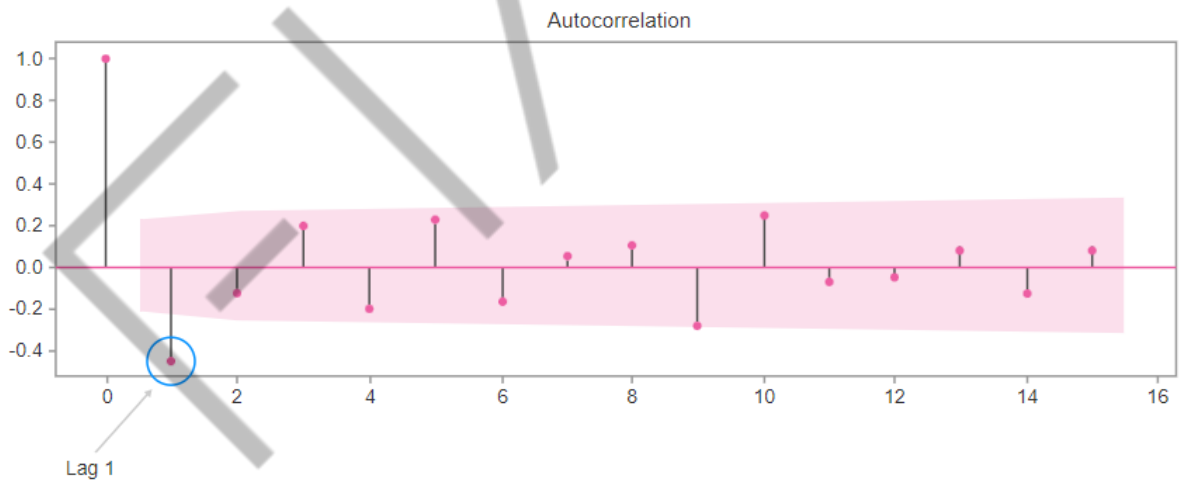

Figura 15 – Gráfico ACF série estudada
Fonte: Elaborado pelo autor (2021)

A análise do gráfico é similar ao exemplo do AR, o objetivo é descobrir pontos fora da área tracejada. Nesse caso, observamos somente o primeiro *lag* fora da linha. Portanto, a ordem MA será 1.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.5 Treinamento do modelo ARIMA

Após determinar todas as ordens do modelo, finalmente poderemos realizar o treinamento. Diante disso, o primeiro passo é dividir as amostras de treino e teste; contudo, em séries temporais, não podemos dividir aleatoriamente as amostras. Devemos selecionar um passado e buscar prever o futuro.

No Código-fonte “Divisão treino e teste”, apresentamos uma forma de dividir as amostras. No exemplo, separaremos sete dias no futuro para testar a capacidade preditiva do modelo.

```
# tamanho amostra teste
test_size = 7

# divisão dos dados
dados_treino = dados_analises[:-test_size]
dados_teste = dados_analises[-test_size:]
```

Código-fonte 12 – Divisão treino e teste
Fonte: Elaborado pelo autor (2021)

Uma vez que os dados foram separados, podemos treinar o algoritmo. Neste momento, avaliaremos o modelo ARIMA(4, 1, 0). Isso indica ordem de integração I(1), AR(4) e MA(0); neste momento, vamos avaliar se precisaremos adicionar o componente MA(1) em uma próxima etapa ou não. No Código-fonte “Modelo ARIMA treinado”, apresentamos o processo para obter as estimativas.

```
# modelo
modelo_4_1_0 = ARIMA(dados_treino['qtd_inscricoes'].values, order=(4,
1, 0), trend = 'n').fit()

# resultados
modelo_4_1_0.summary()
```

Código-fonte 13 – Modelo ARIMA treinado
Fonte: Elaborado pelo autor (2021)

Alguns detalhes merecem ser explicados. O comando order=(4,1,0) auxilia a definir a ordem do modelo e a opção “trend=’n’” significa que o modelo não precisa calcular a constante. Na Figura “Resultados ARIMA”, demonstramos os resultados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SARIMAX Results

Dep. Variable:	y	No. Observations:	70
Model:	ARIMA(4, 1, 0)	Log Likelihood	-232.880
Date:	Sat, 13 May 2023	AIC	475.759
Time:	12:26:13	BIC	486.930
Sample:	0	HQIC	480.191
	- 70		
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	-0.7987	0.107	-7.489	0.000	-1.008	-0.590
ar.L2	-0.7764	0.144	-5.410	0.000	-1.058	-0.495
ar.L3	-0.5104	0.148	-3.454	0.001	-0.800	-0.221
ar.L4	-0.5097	0.144	-3.540	0.000	-0.792	-0.227
sigma2	48.6514	5.788	8.406	0.000	37.307	59.995

Coeficientes

p-values

Ljung-Box (L1) (Q):	0.01	Jarque-Bera (JB):	110.77
Prob(Q):	0.94	Prob(JB):	0.00
Heteroskedasticity (H):	5.23	Skew:	1.74
Prob(H) (two-sided):	0.00	Kurtosis:	8.15

Figura 16 – Resultados ARIMA
Fonte: Elaborado pelo autor (2021)

Podemos destacar os valores dos coeficientes e os p-values. Os p-values verificam se as defasagens utilizadas foram significantes. Como todos os valores foram menores que 0,05, indica-se que as ordens AR foram importantes no treinamento do modelo.

Percebemos que todos os coeficientes foram negativos, isso indica que, caso as vendas diminuam em um dia, subirão no próximo, ou se as vendas aumentarem em um dia, no próximo dia espera-se uma redução.

Uma vez que o algoritmo foi treinado, precisamos verificar se o modelo ficou bem ajustado na amostra de treino. Esse processo é realizado pela observação dos gráficos ACF e PACF dos erros do modelo. No Código-fonte “Obtenção dos gráficos ACF e PACF dos erros”, apresentamos o processo.

```
# acf
fig = plt.figure(figsize=(12,8))
ax1 = fig.add_subplot(211)
fig = plot_acf(modelo_4_1_0.resid, lags=15, ax=ax1)

# pacf
fig = plt.figure(figsize=(12,8))
ax1 = fig.add_subplot(211)
fig = plot_pacf(modelo_4_1_0.resid, lags=15, ax=ax1)
```

Código-fonte 14 – Obtenção dos gráficos ACF e PACF dos erros
Fonte: Elaborado pelo autor (2021)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Um ponto a ressaltar é que os gráficos foram obtidos utilizando os dados “modelo_4_1_0.resid”, esses dados são os erros do algoritmo na amostra de treino. Na Figura “Gráfico ACF erros” e “Gráfico PACF erros”, temos os resultados.

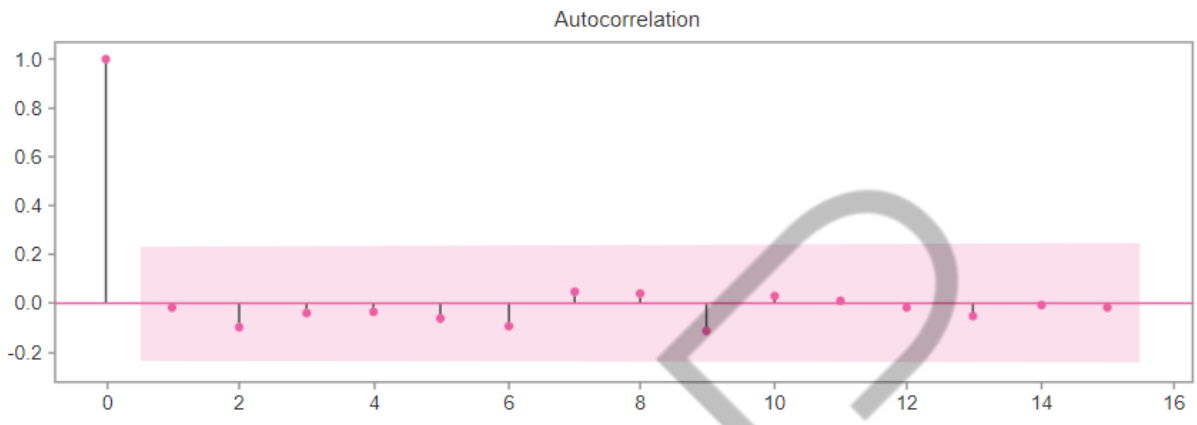

Figura 17 – Gráfico ACF erros
Fonte: Elaborado pelo autor (2021)

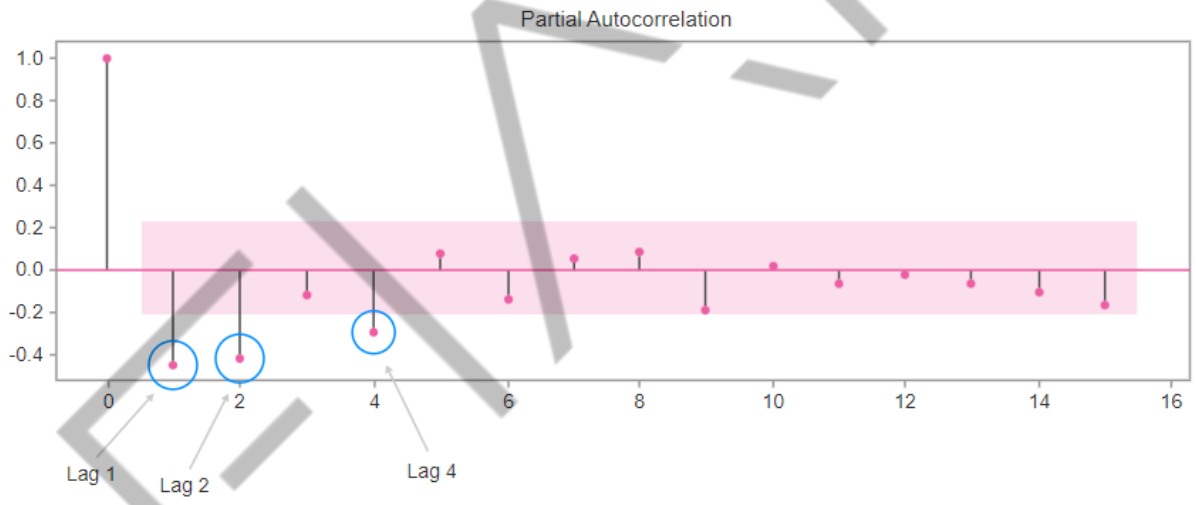

Figura 18 – Gráfico PACF erros
Fonte: Elaborado pelo autor (2021)

Pela análise dos gráficos, percebemos que nenhum ponto esteve fora da área marcada, indicando que nenhum outro dado do passado da série será importante para o treinamento do algoritmo. Diante disso, podemos suprimir a ordem MA(1), uma vez que não temos mais erros que possuem correlação com a série estudada.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.6 Avaliação do modelo

Uma vez que o modelo foi treinado, resta somente avaliar os resultados na amostra de teste. Em nosso exemplo, somente separamos sete dias no futuro para avaliação. Em casos do mundo real, em que temos mais dias para treinar, pode ser possível testar 30 a 60 dias à frente.

A primeira etapa para avaliar a amostra de teste é realizar as previsões do modelo. No Código-fonte “Obtenção das previsões do modelo”, apresentamos o processo para obter as projeções para os próximos sete dias.

```
# obter previsoes
previsoes_ajustadas = modelo_4_1_0.predict(start = 70, end = 76,
dynamic=False)


# adicionar no dataframe
dados_teste['previsoes'] = previsoes_ajustadas
dados_teste
```

Código-fonte 15 – Obtenção das previsões do modelo
Fonte: Elaborado pelo autor (2021)

Como o modelo foi estimado utilizando as primeiras diferenças, as previsões também serão nessa unidade de medida. Diante disto, precisamos realizar o ajuste no código para obter as medidas próximas aos dados originais. Na Figura “Comparação dados teste e previsões”, apresentamos os dados reais da amostra de teste contra as previsões.

	qtd_inscricoes	previsoes
data_venda		
2017-09-04	27	21.644470
2017-09-05	35	20.229059
2017-09-06	32	26.492402
2017-09-08	14	25.319507
2017-09-11	63	22.806571
2017-09-12	26	23.248914
2017-09-13	3	22.253003

Figura 19 – Comparação dados teste e previsões
Fonte: Elaborado pelo autor (2021)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Além da observação direta das diferenças entre dados reais e previsões, podemos obter as métricas de avaliação: RMSE, MAE e MAPE. No Código-fonte "Obtenção das medidas de avaliação amostra teste", apresentamos como obter as métricas de avaliação do modelo na amostra de teste.

```
# separar os dados
dados_reais = dados_teste['qtd_inscricoes'].values

# obter rmse
rmse = np.sqrt(mean_squared_error(dados_reais, previsoes_ajustadas))
rmse

# obter mape
mape = mean_absolute_percentage_error(dados_reais, previsoes_ajustadas
)
mape

# obter mae
mae = mean_absolute_error(dados_reais, previsoes_ajustadas)
mae
```

Código-fonte 16 – Obtenção das medidas de avaliação amostra teste
Fonte: Elaborado pelo autor (2021)

Os resultados apresentam os valores RMSE = 18,51, MAPE = 1,25 e MAE = 14,16. Esses valores sozinhos não trazem tanta informação, porém, caso um segundo modelo seja aplicado, podemos comparar essas decorrências e definir qual algoritmo melhor trouxe resultados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

ATIVIDADE TREINO – ANÁLISE DE SÉRIES TEMPORAIS

Essa atividade não é avaliativa, caso queira ter um feedback, entre em contato com seu tutor pelo Teams

O arquivo “dados_bolsa.csv” foi disponibilizado, vamos utilizar as ferramentas para treinar um modelo de série temporal.

Ao final, deve ser entregue um arquivo “.ipynb” contendo os códigos e respostas.

1. Acessar o *Colab*, carregar os módulos e os dados necessários.

2. Realizar uma preparação de dados:

   a) Filtrar os nulos.

   b) Ajustar a data.

   c) Selecionar as colunas “data” e “petr4”.

   d) Definir “data” como índice do dataframe.

3. Analisar o gráfico da série e testar estacionariedade.

4. Treinar o modelo ARIMA conforme o seguinte:

   a) Avaliar parâmetros AR.

   b) Avaliar parâmetros MA.

   c) Dividir em treino e teste. Como teste, deixar somente cinco linhas.

Além de treinar, apresentar o sumário dmodelo (exemplo: modelo_treinado.summary()).

5. Avaliar a qualidade do modelo na amostra de treino.

6. Avaliar o modelo na amostra de teste.

Importante: respostas no arquivo “.ipynb” disponibilizado.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

BOX, G. M.; JENKINS, G. M.; REINSEL, G. C. **Time Series Analysis**. 4. ed. New Jersey: John Wiley & Sons, 2008.

SHUMWAY, R. H.; STOFFER, D. S. **Time series analysis and its applications with R examples**. Nova York: Springer, 2017.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com